

Zentist: RPA Engineer (Lead) — Test Task

Stack (must): Python 3.11+, Playwright, Pytest + up to you

Time: 3–5 days

AI usage: allowed.

Context

The company automates work with a large number of external portals (~100: customer accounts, registries, HR and banking systems). Some portals are similar to each other, some are structured in fundamentally different ways. There is a set of shared connectors — reusable blocks independent of any specific portal (download a file, parse it, persist to a database, notify an administrator, and so on).

Portals are processed on a schedule in unattended mode, with no human watching; some portals run several times a day to keep their data current. The scale to design for: on the order of **a few hundred jobs per day** (roughly one per portal), and across all of them about **30,000 work items processed daily** — each job logs into its portal and works through its own batch of items (accounts, employees, records). The system must accept this stream of jobs and route them to the right bots.

No starter repository is provided: structuring the project is up to you. What you build for two portals must be a foundation that credibly extends to this scale.

Part 1 — Code

Implement automation for **two different portals** on a shared codebase.

Portal A — OrangeHRM

<https://opensource-demo.orangehrmlive.com>

Login: Admin / admin123 (single login). The work happens *inside* the portal, against employee records.

*(*disclaimer, data on the portal may be reset to defaults after some time)*

The bot is driven by an input set of employee records — a mix of people who are **already in the system** and people who are **not yet in it**. You design the input format and contents. The employee directory spans several pages and hundreds of records; a single employee must be located within it.

For each employee in the input:

- Determine whether the employee already exists in the system.
- If they do not exist, add them (Add Employee), then locate the newly created record.
- For every employee — newly added or already existing — open the record and bring its Job section up to date: set/refresh **Job Title** and **Employment Status** to the target values from the input.
- In the **Salary** section, generate a text document containing that employee's salary details and upload it as an attachment — but only if the employee does not already have one.

This bot runs several times a day. Each run must leave every processed employee's Job Title, Employment Status and attachment matching the intended state — regardless of how many times the bot has already run that day.

Portal B — Sauce Demo

<https://www.saucedemo.com>

Process **all of the following accounts**, password `secret_sauce` for every one:

`standard_user`, `locked_out_user`, `problem_user`, `performance_glitch_user`,
`error_user`, `visual_user`

For each account, complete a full purchase end to end: log in, add **three items** to the cart, go through checkout (entering the required customer information), reach the order confirmation, capture the resulting order details, persist them, and finish the order.

Every account on Sauce Demo and every employee on OrangeHRM must be processed. They do not all behave the same way — some will not complete the scenario cleanly. The bot must process each one, record a per-item outcome (success / failure and the reason), and still produce a coherent report when some do not succeed.

Requirements for the code

- An explicit, defensible separation between logic shared across all portals and logic specific to a single portal. Adding a third portal must not require touching the first two.
- Shared connectors genuinely reused by both bots, not duplicated per portal.
- Bot results persisted through shared connectors into a database (SQLite at minimum), with a per-item outcome for everything processed.
- After a run, a summary report is generated and sent by e-mail (recipient configured outside the code).
- The system survives an unstable portal — slow responses, a failing step, a dropped session — with retries/timeouts handled in a reusable layer, not copy-pasted per bot.
- A failure on one account does not abort the rest of the batch.
- Re-running on the same day does not duplicate results or corrupt the report.
- Failed or skipped accounts are recorded with a reason; a partial run still produces a coherent report.
- The system emits enough operational signal to be monitored: structured logs and run/account metrics. A human must be able to answer “what happened in today’s run?” without reading raw stack traces.
- Credentials are not stored in the repository or in plaintext in code, and nothing sensitive leaks into logs or the report.
- Configuration (e-mail recipient, DB location, secrets source, any limits) lives outside the code.

Deliverables for Part 1

- A repository with an open PR to master containing all changes.
 - `README.md`: how to run locally, how it is configured, how a new portal is added.
 - A CI/CD pipeline (GitHub Actions or equivalent) running on the PR: installs dependencies, runs linting and the test suite. Set it up as you would for production.
 - Tests (Pytest) covering what you consider worth covering.
-

Part 2 — Design essay

A written document (DESIGN.md, ~1–3 pages) describing how you would run this in production **at full scale: a few hundred daily jobs across ~100 portals, processing ~30,000 work items per day in total.**

Use diagrams wherever they make the design clearer — include whatever diagrams you consider necessary (architecture, data flow, sequence, deployment — your choice). They will be discussed.

Address at least:

- **Orchestration & throughput.** How a few hundred daily jobs — together covering ~30,000 work items — are scheduled, distributed and processed within a daily window. What runs concurrently, and how concurrency is bounded.
- **Per-portal constraints.** How you respect a portal's own limits (one session per login, rate limits, lockout after failed attempts) while a single job may need to push hundreds or thousands of items through that one portal.
- **Isolation & failure.** How a single failing or hanging portal is prevented from stalling or crashing the rest. What happens to the accounts already processed when a job dies partway through its batch.
- **Monitoring & observability.** What you would monitor across ~100 portals and ~30,000 daily items, and which tools you would use. Cover at least: how a failed or stuck job is detected and surfaced, and to whom; what metrics and dashboards you would track; how alerting is set up so a 3-person team learns about a problem before the business does; and how you would catch a silent failure, where a bot reports success but has been collecting the wrong data after a portal changed its layout. Name concrete tooling and why.
- **Idempotency & recovery.** What a same-day re-run does, and how you avoid re-processing the accounts a job already handled before it failed.
- **Technology choices.** The job-delivery mechanism, concurrency model (async / threads / processes / workers), scheduler, and infrastructure you would use — for each, what you chose, what you rejected, and why, with this scale in mind.
- **Operating it.** How a 3-person team monitors and triages 100 portals, how credentials are stored and rotated at this volume, and how the system degrades when a portal is down through no fault of yours.